

Formal Verification: Vouch-Based Trust Model

Zach Kelling, Lux Industries

December 2025

Abstract

We formalize the vouch-based trust model where trust flows through attestation chains without any global root or certificate authority. Each vouch bounds the scope of delegated authority, and chain authority is monotonically decreasing.

1 Introduction

The vouch-based trust model replaces traditional PKI with attestation chains. No CA, no global root. Trust flows through signed vouches, each bounding the scope of delegated authority. This model is used by Hanzo IAM, Lux validator trust, and Zoo role endorsements.

2 Definitions

Definition 1 (RecognitionLevel). *Trust levels: unrecognized (0), vouched (1), direct (2).*

Definition 2 (Vouch). *A vouch: voucher, vouchee, scope bound, expiry timestamp.*

3 Theorems and Axioms

Theorem 3 (unrecognized_min). *Unrecognized is the minimum recognition level.*

Theorem 4 (chain_authority_monotone). *Adding a vouch to a chain can only reduce authority: $\text{chainAuthority}(v :: \text{chain}, \text{top}) \leq \text{chainAuthority}(\text{chain}, \text{top})$.*

Proof. By induction on the chain, using min monotonicity. □

Theorem 5 (empty_chain_full). *Empty chain gives full authority: $\text{chainAuthority}([], \text{top}) = \text{top}$.*

Theorem 6 (chain_bounded). *Chain authority is bounded by the top: $\text{chainAuthority}(\text{chain}, \text{top}) \leq \text{top}$.*

Theorem 7 (chain_bounded_by_scope). *Chain authority is bounded by every vouch's scope in the chain.*

Proof. By induction: each vouch's scope constrains the min fold. □

4 Lean Source

```
/-
  Vouch-Based Trust Model

  No CA. No global root. Trust flows through attestation chains.
  Each vouch bounds the scope of delegated authority.

  Maps to:
  - hanzo/iam: attestation chains
  - lux/node: validator vouch graph
  - zoo/contracts: role endorsements

  Properties:
  - Chain authority is monotonically decreasing
  - Unrecognized keys have no authority
  - Vouch signatures bind ALL security-relevant fields
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Data.List.Basic
import Mathlib.Tactic

namespace Trust.Vouch

/-- Recognition level: how much we trust this key -/
inductive RecognitionLevel where
| unrecognized -- No trust path
| vouched      -- Vouched by someone trusted
| direct       -- Directly trusted (root)
deriving DecidableEq, Repr

def RecognitionLevel.rank : RecognitionLevel → Nat
| .unrecognized => 0
| .vouched      => 1
| .direct       => 2

/-- Unrecognized is minimum recognition -/
theorem unrecognized_min (r : RecognitionLevel) :
  RecognitionLevel.rank .unrecognized ≤ RecognitionLevel.rank r := by
  cases r <|> simp [RecognitionLevel.rank]

/-- A vouch: one key vouches for another with bounded scope.
  The signature MUST bind all fields to prevent replay attacks. -/
structure Vouch where
  voucher : Nat      -- voucher's public key (abstract)
  vouchee : Nat      -- who is being vouched for
  scope   : Nat      -- authority bound (abstract)
  expires : Nat      -- expiry timestamp

/-- Vouch chain authority: foldl min over scope values.
  Authority can only NARROW through delegation. -/
def chainAuthority (chain : List Vouch) (top : Nat) : Nat :=
  chain.foldl (fun acc v => min acc v.scope) top
```

```

/-- Adding a vouch can only reduce authority -/
theorem chain_authority_monotone (chain : List Vouch) (v : Vouch) (top : Nat)
:
  chainAuthority (v :: chain) top  chainAuthority chain top := by
simp [chainAuthority, List.foldl_cons]
induction chain generalizing top with
| nil => simp [List.foldl, chainAuthority]; exact Nat.min_le_left top v.
  scope
| cons hd tl ih =>
  simp [List.foldl]
  apply ih

/-- Empty chain gives full authority -/
theorem empty_chain_full (top : Nat) :
  chainAuthority [] top = top := by
simp [chainAuthority]

/-- Chain authority is bounded by top -/
theorem chain_bounded (chain : List Vouch) (top : Nat) :
  chainAuthority chain top  top := by
induction chain generalizing top with
| nil => simp [chainAuthority]
| cons hd tl ih =>
  simp [chainAuthority, List.foldl_cons]
  calc chainAuthority tl (min top hd.scope)
    min top hd.scope := ih (min top hd.scope)
    _ top := Nat.min_le_left top hd.scope

/-- Chain authority is bounded by every vouch's scope -/
theorem chain_bounded_by_scope (chain : List Vouch) (top : Nat) (v : Vouch)
(hv : v ∈ chain) :
  chainAuthority chain top  v.scope := by
induction chain generalizing top with
| nil => exact absurd hv (List.not_mem_nil _)
| cons hd tl ih =>
  simp [chainAuthority, List.foldl_cons]
  cases List.mem_cons.mp hv with
| inl h =>
  subst h
  calc chainAuthority tl (min top hd.scope)
    min top hd.scope := chain_bounded tl _
    _ hd.scope := Nat.min_le_right top hd.scope
| inr h => exact ih (min top hd.scope) h

end Trust.Vouch

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Trust/Vouch.lean>

White papers: <https://papers.lux.network>

Related white papers:

- lux-id-iam.tex

- lux-id-did-specification.tex